HN 每日一片 · 2026-11-10

從零手刻 micrograd：看懂反向傳播

The spelled-out intro to neural networks and backpropagation: building micrograd | Andrej Karpathy

HN 推薦人數 17　跨 16 個月被推薦　影片：https://www.youtube.com/watch?v=VMj-3S1tku0

> **Karpathy 用約 100 行 Python 從零寫出自動微分引擎 micrograd，一步步示範反向傳播（backpropagation）與神經網路訓練的原理。**

為什麼值得看：HN 工程師認為這是理解反向傳播與神經網路的最佳起點：Karpathy 用直覺講清楚「為什麼」，並且全程用程式碼從頭建起，連 PyTorch 的做法也對照過。

重點整理 KEY POINTS

00:32　micrograd 是什麼

micrograd 是一個自動梯度（autograd）引擎，核心功能是反向傳播：有效率地算出損失函數（loss function）對神經網路權重的梯度，再反覆微調權重來降低損失。這是 PyTorch、JAX 這類函式庫的數學核心。它是純量（scalar）版本，把神經網路拆到單一數字的加與乘，教學用而非生產用；換成張量（tensor）只是為了平行運算加速，數學不變。

07:03　整個引擎只有約 100 行

Karpathy 的主張是：訓練神經網路真正需要的就是 micrograd，其餘都只是效率問題。engine 檔（自動微分引擎）約 100 行簡單 Python，nn.py（神經元、層、多層感知器 multi-layer perceptron）再加約 50 行，合計約 150 行。他認為只要看懂這些，就理解了神經網路訓練的本質。

10:52　導數的直覺：微小擾動看反應

他不用符號推導，而是回到導數定義：把輸入 x 加上很小的 h，看輸出變化多少，再除以 h 就是斜率。以 $f(x)=3x^2-4x+5$ 在 x=3 數值估算得斜率 14，在 −3 斜率為負。接著對 d=a×b+c 分別微擾 a、b、c，得到斜率 −3、2、1，與微積分結果一致。h 太小會因浮點數精度出問題。

19:11　建立 Value 物件與運算圖

他寫了 Value 類別包住純量，並用 Python 的 __add__、__mul__ 定義加與乘，讓每個結果記錄來源子節點（_prev）和產生它的運算（_op），形成運算圖。接著加上標籤並用 graphviz 畫出圖，把 a、b、c、f 經過運算得到輸出 L=−8 的前向傳播（forward pass）視覺化。

38:24　鏈鎖律：反向傳播的核心

他稱這是理解全部反向傳播的關鍵節點。加法節點的局部導數是 1，所以只是把上游梯度原樣分給兩個子節點；乘法節點的局部導數是另一個輸入的值。再用鏈鎖律（chain rule）：整體梯度 = 局部導數 × 上游梯度。他以「汽車速度是腳踏車 2 倍、腳踏車是步行 4 倍，所以汽車是 8 倍」說明為何要相乘，並用數值微擾逐一驗證手算梯度。

52:50 **單一神經元與 tanh 反向傳播**

神經元模型是：輸入乘權重、加偏差（bias）、再過 tanh 這類壓縮用的活化函數（activation function）。tanh 的局部導數是 $1 - \tanh^2$ ，他手動走完整個反向傳播，並指出輸入為 0 時對應權重梯度為 0（乘以零就不會有影響）。他也強調運算顆粒度可自選：只要能寫出局部導數，複合運算也能直接當一個節點。

1:09:34 **自動化 backward 與拓撲排序**

他把手動步驟寫進每個節點的 _backward 閉包函式，再用拓撲排序（topological sort）確保一個節點要等所有依賴它的節點都處理完才反傳，最後包成 backward()。接著修掉一個 bug：同一變數被用多次（如 a+a）時梯度會被覆蓋，正確做法是用 += 累加梯度。之後補上 exp、次方、除法、減法，並證明拆成 exp 版的 tanh 結果完全相同。

1:39:28 **對照 PyTorch 與組裝 MLP**

他用 PyTorch 寫同一個神經元，前向 0.707、梯度 0.5、0、−1.5、1 完全一致，差別只在 PyTorch 用張量加速。接著按 PyTorch 的 API 風格寫 Neuron、Layer、MLP 三個類別，組成輸入 3、兩層各 4、輸出 1 的網路，並用它跑四筆資料的簡單二元分類範例。

1:51:36 **損失函數與梯度下降**

他用均方誤差（mean squared error）把預測與目標的差距壓成單一數字 loss，初始約 7。呼叫 loss.backward() 後，每個參數都有梯度；沿梯度反方向微調參數（要加負號，因為要降 loss），反覆進行就是梯度下降（gradient descent）。學習率太大會越過頭而不穩，太小則收斂慢。這個網路共有 41 個參數。

2:10:28 **經典 bug：忘了 zero grad**

他坦承犯了常見錯誤：每次更新前沒把梯度歸零，導致梯度不斷累加。因為問題太簡單，網路還是收斂，反而像放大了步長。修正方式是在 backward 前把每個參數的 grad 設回 0，也就是 PyTorch 的 zero_grad。他以此提醒神經網路的 bug 常常「悄悄地沒壞」，較複雜的問題就會訓練失敗。

對你的啟發 FOR MILLER

- 做設計管理時，你可以借用他的做法：先用最小、最簡單的版本把原理講透（純量而非張量），再談規模與效率。向跨部門或客戶說明複雜流程時，先給一個看得懂的小例子，比直接給完整系統更有說服力。
- 「微擾一點點，看結果變多少」就是敏感度分析（sensitivity analysis）。看台股模型或持倉時，可以問：成本、營收或利率各動一點，報酬會怎麼變？梯度的概念能讓你判斷哪個變數最值得盯，而不是每項都平均用力。
- 學習率過大會讓損失暴衝，過小則進展太慢。這對應到加碼與調整持倉的步幅：一次動太大可能越過頭而失控，動太小又錯過機會。影片也提醒：系統「看起來有效」不代表沒有 bug，要用獨立的檢查（如數值梯度檢查）驗證。

HN 網友怎麼說 FROM HACKER NEWS

> 我強烈推薦 Andrej Karpathy 的這個 YouTube 播放清單。
> “I highly recommend this youtube playlist from Andrej Karpathy”
> — jwitthuhn · 2025-12 · https://news.ycombinator.com/item?id=46359861

> Andrej Karpathy 那堂很棒的 micrograd 課。
> “Andrej Karpathy's awesome micrograd lecture”
> — omrishn · 2025-11 · https://news.ycombinator.com/item?id=45964192

> 其實是 Karpathy 在教我們，讓我們能理解它們並做出自己的版本。
> “its actually Karpathy who teaches us :) so we can understand them and make our own.”
> — jackdoe · 2025-06 · https://news.ycombinator.com/item?id=44210120

> 最好的入門系列之一（就算先不管神經網路的部分也是）。
> “One of the best introductory series (even if you ignore the neural net part of it)”
> — throwaway7783 · 2025-05 · https://news.ycombinator.com/item?id=43927931

> 去看 Karpathy 最近的課，那些都是金礦。
> “Watch Karpathy's recent lectures. They're gold.”
> — paulsmith · 2023-09 · https://news.ycombinator.com/item?id=37356355

名詞小抄 GLOSSARY

backpropagation（反向傳播）	從輸出往回，逐層用鏈鎖律算出每個參數對損失的影響。
gradient（梯度）	某個值微幅增加時，輸出會跟著增加多少的斜率。
chain rule（鏈鎖律）	把一層層的局部變化率相乘，得到整體變化率。
loss function（損失函數）	用一個數字衡量預測離目標有多遠，越小越好。
gradient descent（梯度下降）	沿梯度反方向反覆微調參數，讓損失逐步下降。
topological sort（拓撲排序）	把圖的節點排序，使每個節點都排在其依賴者之前。

整理：Claude（依影片字幕撰寫，未看畫面）· 選片資料：HackerTube（hackertube.org，ODC-By 1.0）